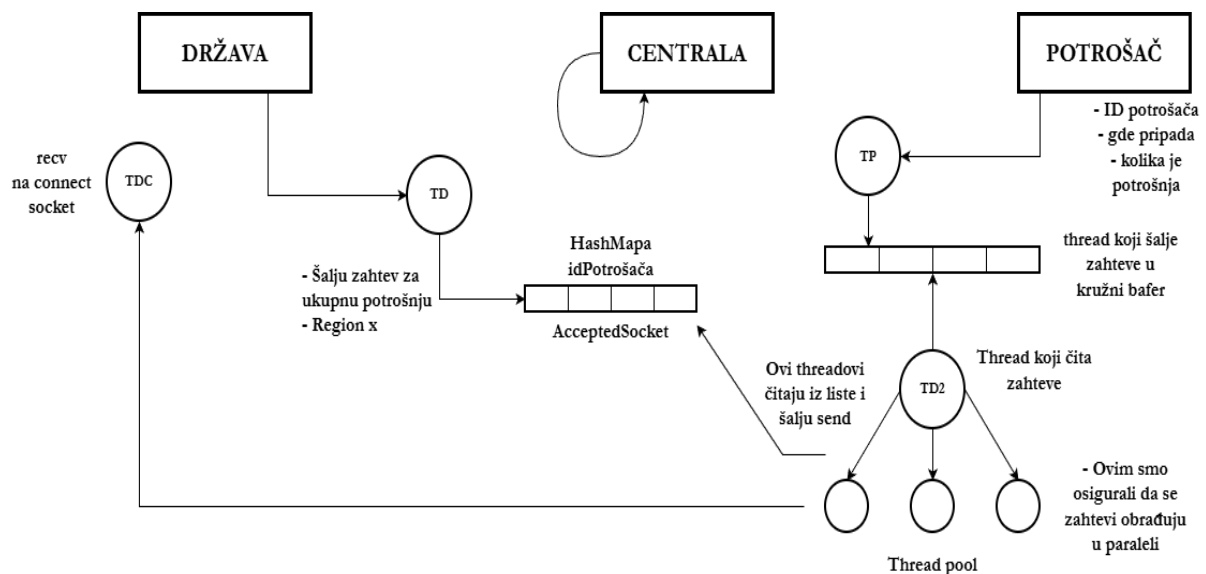


Uvod

Distribuirani servis za agregaciju potrošnje struje: server (Agregator) i klijenti (AgregatorClient) kao potrosaci.

Klijenti se povezuju preko mreze, registruju se i na zahtev šalju potrošnju. Server sve sabira u stablastoj topologiji i prikazuje ukupnu potrošnju. Podržani su dva rezima automatic i batch, i testiranje sa vecim brojem zahteva koji salju klijenti u cilju pracenja memorije.

Dizajn projekta



Centralni agregator i čvorovi u stablu imaju istu strukturu. Root (država) je izvor podataka; ostali cvorovi agregiraju prema nadređenom. Svaki agregator tretira svoje potrosace isto – bez razlike. Server ima prijemnu nit koja prima nove potrosace, registruje ih i cuva u mapi (idPotrosaca → soket) radi brzog pristupa. Kada korisnik pokrene Automatski / Batch / Subtree rezim, sistem koristi thread pool za paralelno slanje zahteva i prijem potrošnje od svih povezanih potrošača. Odgovori se stavljaju u kružni bafer; posebna consumer nit ih čita i agregira kroz stablo.

Topologija mreze drži stablo cvorova u mapi radi brzog pristupa; svaki potrosac ima mapiran nadredjeni agregator .

Cilj dizajna: paralelna obrada više klijenata preko thread poola i kružnog bafera, uz jednostavnu hijerarhijsku agregaciju u memoriji, bez distribuiranih agregatora – samo server i krajnji potrošači.

Struktura podataka

HashMapa

Hash mapa je izabrana zbog brzog pristupa elementima. U projektu se koristi hash mapa u klasi Network:

consumerIdToParent_ – koristi se za brz pristup nadređenom čvoru potrošača pomoću ključa consumerId. Čuva preslikavanje ID potrošača (npr. 10–15) u agregator čvor koji mu je roditelj. Služi za određivanje ko pripada kom delu stabla, validaciju ID-ja potrošača i proveru da li je potrošač u datom podstablu.

Kružni Bafer

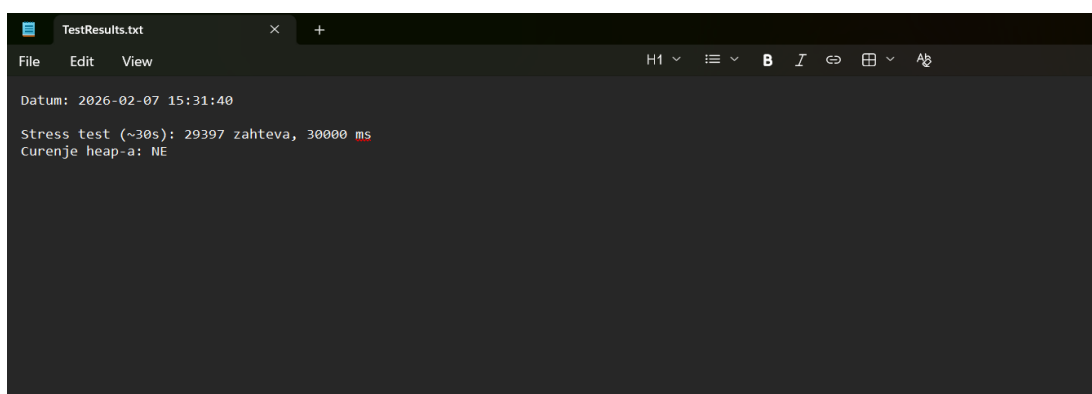
Implementiran je kružni buffer: fiksna veličina, indeksi head i tail se kružno pomicu po nizu, pa se ista memorija ponovo koristi kada se elementi izvade — bez realokacije i pomeranja podataka. U red smestamo ConsumptionReport u kojem se nalaze ID potrosaca i value (vrednost potrosnje, npr. elektricne struje). Kružni buffer služi kao medjubuffer između čvorova u mrezi: jedan deo koda stavlja izvestaje potrosnje, drugi ih uzima na red, uz međusobno zaključavanje i signalizaciju kada ima podataka.

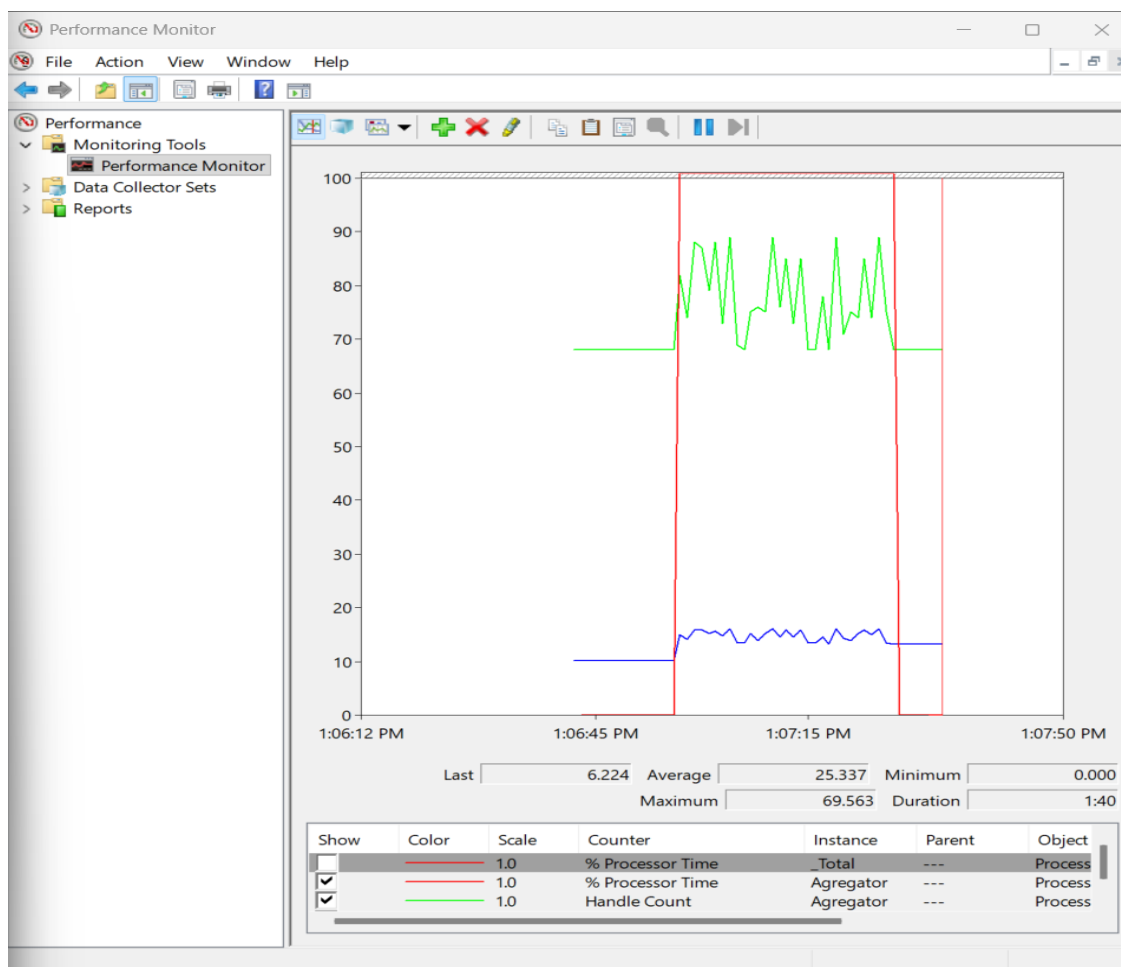
Testiranje

Testovi su radjeni na hijerarhijskoj topologiji mreze sa root cvorom (drzava), agregatorima (Agr 0–5) i potrosacima (10–15). Na svaki agregator su povezani potrosaci putem AgregatorClient procesa. Za testiranje se koristi Stress test (opcija 9 u meniju): for petlja u trajanju od ~30 sekundi u kojoj server u svakoj iteraciji salje zahteve (REQUEST) povezanim klijentima, klijenti vraćaju potrošnju (CONSUMPTION), a server agregira kroz stablo uz thread pool i kružni bafer. Ovim se generiše veliki broj zahteva (reda hiljada ili desetina hiljada) i opterecuje server u zadatom vremenskom intervalu. CRT debug heap koristi se za snimak stanja heap-a pre i posle testa te proveru curenja memorije. Rezultati (broj zahteva, vreme u ms, da li postoji curenje heap-a) upisuju se u TestResults.txt. Svrha testiranja je provera performansi i stabilnosti pri većem broju zahteva i praćenje ispravnog korišćenja memorije u konkurentnom izvršavanju.

Rezultati

Za pracenje memorije koristili smo performance monitor i rezultate smo ispisali u TestResults.txt

A screenshot of a text editor window titled 'TestResults.txt'. The window has a dark theme and a menu bar with 'File', 'Edit', and 'View'. The text content shows the date and time 'Datum: 2026-02-07 15:31:40', followed by 'Stress test (~30s): 29397 zahteva, 30000 ms' where '30000 ms' is in red, and 'Curenje heap-a: NE'.



Iz ovoga vidimo da je sistem stabilan i pod ogromnim opterećenjem, nema curenja i sve je oslobođeno na kraju.

Potencijalna unapredjenja

Port, broj niti u thread poolu i intervali su u kodu; korisno bi bilo izvući ih u config fajl ili argumente komandne linije radi lakšeg podešavanja bez rekompajliranja.

Što se tiče bezbednosti, protokol je čist tekst, bez šifrovanja i autentifikacije, TLS (kriptografski protokol za osiguravanje privatnosti) i provera identiteta klijenta (npr. token ili sertifikat) povećali bi sigurnost.

Takodje u trenutnoj implementaciji nemamo podršku za IPV6 pored IPV4 i mogućnost da server sluša na više adresa/portova po potrebi.